To build a professional-grade, voice-to-voice interview, you need to transition from a sequential process (Text -> Text) to a **streaming pipeline**.

In a conversation, human tolerance for delay is roughly **500ms**. To achieve this without expensive APIs, we use "small-but-mighty" local models and an orchestration framework that supports **WebRTC**.

---

1. Stack

This selection prioritizes low latency (speed) and high accuracy for an interview context.

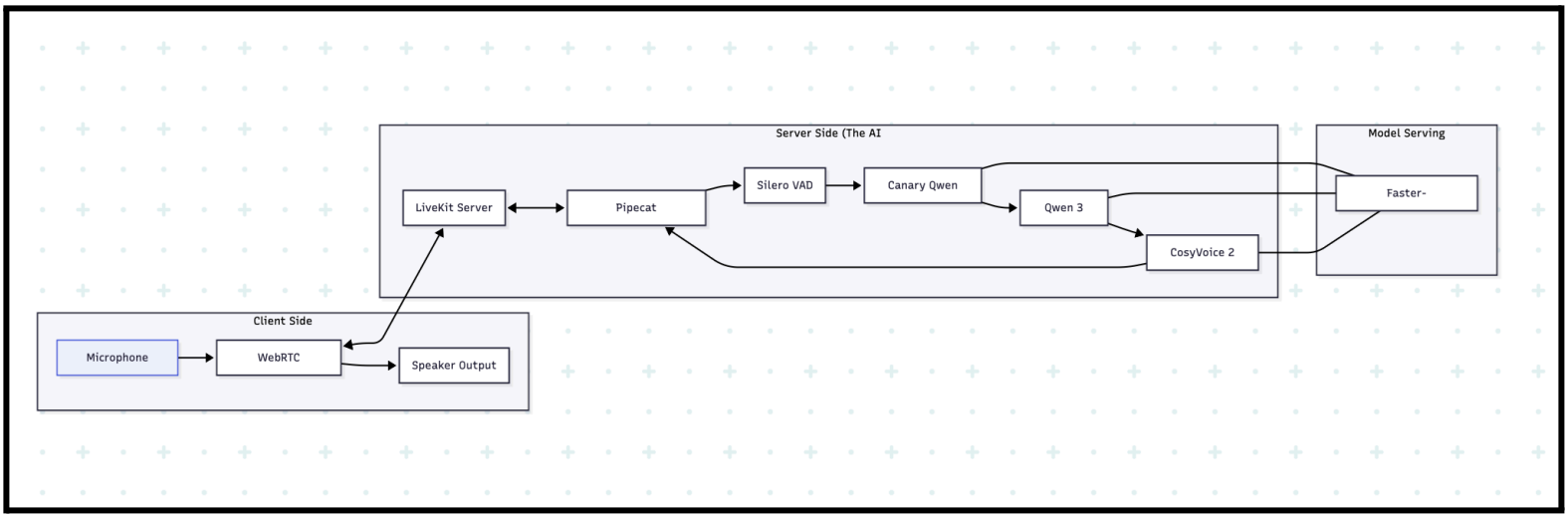
Component	Industry Best (Open Source)	Key Reason for Choice
Transport / Infra	**LiveKit Agents**	The industry standard for WebRTC audio/video. Handles noise and jitter.
Orchestrator	**Pipecat**	Best-in-class Python framework for managing the "Turn-taking" logic.
STT (Listening)	**Canary Qwen 2.5B**	2026's leader for English accuracy (5.6% WER), beating Whisper Large.
Brain (Thinking)	**Qwen 3 - 7B Instruct**	Highly optimized for dialogue and professional "thinking" personas.
TTS (Speaking)	**CosyVoice 2 (0.5B)**	Ultra-low **150ms** streaming latency with human-level emotion.

---

2. Technical Architecture Diagram

The system operates as a **circular stream**. Data flows constantly rather than waiting for files to save.

None



---

3. Detailed Component Breakdown

I. The Orchestrator (Pipecat + LiveKit)

Traditional code uses `input()` and `print()`. For voice, you need **event-driven** code.

- **VAD (Voice Activity Detection):** Instead of waiting for a button, the AI "listens" for silence.
- **Interruption Handling:** If the AI is speaking and the candidate starts talking, the Orchestrator sends a `SIGKILL` to the TTS buffer immediately to make the AI "stop and listen."

II. The Pipeline Logic

To make the AI feel "smart," we use a **Streaming LLM**.

- As soon as the LLM generates the first 3-5 words, those words are sent to the TTS.
- The AI starts speaking the beginning of the sentence while it is still "thinking" of the end. This masks the processing time.

III. Deployment Environment

To run this locally without "Pay-per-use," you need a machine with:

- **GPU:** Minimum NVIDIA RTX 3090 (24GB VRAM) or RTX 4090.
- **Serving Engine:** Use **vLLM** for the LLM and STT. It is 10x faster than standard Python implementations.

---

4. Implementation Blueprint (Python)

Using **Pipecat**, your core server logic would look like this:

Python

```
None


import asyncio
```

```
from pipecat.services.qwen import QwenLLMService
from pipecat.services.cosyvoice import CosyVoiceTTSService
from pipecat.pipeline.pipeline import Pipeline
from pipecat.transports.services.livekit import LiveKitTransport

async def main():
    # 1. Setup Transport (WebRTC)
    transport = LiveKitTransport(room_name="interview-room")

    # 2. Initialize Models
    llm = QwenLLMService(model="qwen3-7b-instruct")
    stt = CanarySTTService() # High-accuracy local STT
    tts = CosyVoiceTTSService(voice="professional_male_1")

    # 3. Define the Interview Persona
    system_prompt = "You are a Senior Technical Recruiter. Conduct a 10-minute
interview."

    # 4. Build the "Continuous Loop" Pipeline
    pipeline = Pipeline([
        transport.input(),   # Listen to Mic
        stt,                 # Convert to Text
        llm,                 # Generate Response
        tts,                 # Convert to Voice
        transport.output()   # Send to User's Speakers
    ])

    await pipeline.run()

if __name__ == "__main__":
    asyncio.run(main())
```